

DAT 6기 캡스톤 프로젝트 보고서

위치 기반 신용카드 혜택 매칭 및 추천 시스템 개발

: LangChain 기반 혜택 데이터 구조화와 위치 기반 앱 서비스 구현 연구

Development of a Location-Based Credit Card Benefit Matching and
Recommendation System

DAT 6기

체리피커

박재행

이명빈

김한별

조준하



Data Analysis & Technology



위치 기반 신용카드 혜택 매칭 및 추천 시스템 개발

:LangChain 기반 혜택 데이터 구조화와 위치 기반 앱 서비스 구현 연구

국문 요약

본 연구는 위치 기반 신용카드 혜택 추천 서비스를 구현하기 위해, 비정형으로 제공되는 카드 혜택 정보를 구조화하고 이를 실제 매장 데이터와 매칭하는 파이프라인을 구축하는 것을 목표로 한다. 먼저 Selenium을 활용하여 카드고릴라 웹사이트의 약 2,900개 카드 상세 페이지에서 혜택·조건·한도 정보를 크롤링하였으며, 소상공인시장진흥공단의 매장 위치 및 업종 데이터를 추가적으로 확보하여 가맹점 기준 정보를 구성하였다.

수집된 혜택 텍스트는 LangChain과 Gemini 2.5 Flash 모델을 통해 가맹점명, 업종(category), 혜택 요약, 전월 실적 조건, 한도(limit), 오프라인 매장 여부 등 표준화된 스키마로 자동 구조화하였다. 이후 개체명 인식(NER) 기반 매칭 컬럼을 생성하여 매장 DB와 혜택 정보를 연계할 수 있는 데이터 기반을 확보하였다.

최종적으로는 Kotlin 기반 Android 앱과 Express.js 서버를 개발하여 지도 기반 매장 시각화, 혜택 매칭도 제공, 카드 관리 기능 등 사용자 중심 UI/UX 프로토타입을 구현하였다. 본 연구는 비정형 혜택 데이터의 구조화와 공간 데이터 매칭을 결합하여 위치 기반 카드 추천 서비스의 실현 가능성을 제시한다.

Abstract

This study aims to develop a location-based credit card benefit recommendation system by structuring unorganized card benefit information and matching it with real-world merchant data. Using Selenium, approximately 2,900 card detail pages from the CardGorilla website were crawled to collect benefit, condition, and limit information. Additionally, merchant location and business category data from the Small Enterprise and Market Service (SEMAS) were incorporated to establish a unified merchant reference dataset.

The collected unstructured text was processed using LangChain and the Gemini 2.5 Flash model to extract standardized fields such as merchant name, category, benefit summary, spending conditions, benefit limits, and location-based applicability. Named Entity Recognition (NER)-style matching columns were then generated to connect the structured benefit data with merchant records stored in a local SQLite database.



Finally, an Android application was developed using Kotlin and Jetpack Compose, supported by an Express.js backend, providing map-based merchant visualization, benefit matching scores, and card management features. This research demonstrates the feasibility of integrating structured LLM-based benefit extraction with spatial data processing to enable a personalized, location-aware credit card recommendation service.

DAT 6기

체리피커

박재행

이명빈

김한별

조준하



목 차

제 1장 서론	1
제 1절 연구 배경	1
제 2장 데이터 소개.....	2
제 1절 카드 혜택 정보 크롤링	2
제 2절 소상공인시장진흥공단 DB 활용	3
제 3장 분석 및 구조화 과정.....	4
제 1절 LangChain 기반 카드 혜택 정보 구조화.....	4
제 2절 LangChain 기반 매칭 컬럼 생성	7
제 3절 SQLite 및 JS 기반 매장 매칭 및 지도 데이터 연계.....	9
제 4장 시스템 구현 결과	11
제 1절 앱 개발(프론트엔드·백엔드)	11
제 2절 UI/UX 및 위치 기반 추천 기능 구현.....	11
제 5장 결론	14
제 1절 연구 결과 종합 및 시사점	14
제 2절 한계점 및 향후 연구 방향	15

참고문헌	16
국내문헌	16

그림 목차

[그림 2.1-1]	2
[그림 3.1-1]	4
[그림 3.1-2]	4
[그림 3.1-3]	5
[그림 3.1-4]	6
[그림 3.1-5]	6
[그림 3.2-1]	7
[그림 3.2-2]	9
[그림 3.3-1]	10
[그림 4.2-1]	12
[그림 4.2-2]	13
[그림 4.2-3]	14



제 1 장 서론

제 1 절 연구 배경

현대사회에서 신용카드는 국민 실생활과 실물경제를 연결하는 핵심 금융 인프라로 자리잡았다. 지난 11 월 20 일 개최된 여신전문금융업권 CEO 간담회 자료에 따르면, 국내 신용카드 시장은 320 만 가맹점을 통해 연간 소비 규모가 1,000 조원에 달하며, 경제활동인구 1 인당 평균 보유 매수는 4.6 매를 기록하고 있다. 이는 다카드 소지의 보편화를 넘어, 신용카드가 개인의 소비생활에서 중추적인 역할을 수행함을 시사한다.¹

이러한 시장의 양적 성장과 금융당국의 소비자 중심 정책 기조에도 불구하고, 실제 소비자가 체감하는 정보 환경은 여전히 불투명하다. 수많은 카드가 제공하는 복잡한 혜택 정보는 웹 상에 비정형 텍스트로 산재해 있어, 소비자가 결제 시점에 최적의 카드를 선택하기 위해서는 높은 정보 탐색 비용을 지불해야 한다. 이는 소비자가 효용을 극대화하는 합리적 선택을 단념하게 만들며, 금융 인프라의 고도화가 소비자의 실편익으로 이어지지 못하는 한계점을 드러낸다.

이에 본 연구는 사용자 위치 정보를 기반으로 소비 성격에 부합하는 최적의 신용카드를 추천하는 시스템을 제안한다. 이를 통해 앞서 제기된 문제를 해결하고 소비에 있어 사용자의 편의와 효용을 증진시키는 것을 목표로 한다.

¹ 금융위원회. (2025). 여신업권 CEO 간담회 모두 말씀 [보도 자료].



제 2 장 데이터 소개

제 1 절 카드 혜택 정보 크롤링

본 연구에서는 국내 주요 카드 정보 포털 <카드고릴라>를 대상으로 약 3,000 개 가량의 신용카드 상세 정보를 수집하였다.

수집 대상 웹페이지들은 초기 로딩 시 혜택의 요약 정보만을 노출하며, 구체적인 혜택 적용 가맹점, 실적 조건 및 한도 등 핵심 데이터는 펼쳐보기 버튼에 숨겨져 있는 구조를 가진다.

이에 본 연구에서는 Python 기반의 Selenium 라이브러리를 활용하여 브라우저 자동화 스크립트를 작성하였다. 해당 스크립트는 페이지 내 혜택 정보 영역에서 확장 가능한 요소를 식별하여 클릭 이벤트를 강제로 발생시킴으로써 전체 혜택 텍스트를 활성화할 수 있게 한다. 이를 통해 카드 상품명과 혜택 원문을 추출하였다.

```
def create_driver():
    chrome_options = Options()
    chrome_options.add_argument("--headless")
    chrome_options.add_argument("--no-sandbox")
    chrome_options.add_argument("--disable-dev-shm-usage")
    chrome_options.add_argument("--lang=ko_KR")
    chrome_options.add_argument("--window-size=1920x1080")

    service = Service(ChromeDriverManager().install())
    driver = webdriver.Chrome(service=service, options=chrome_options)
    driver.implicitly_wait(3)
    return driver

# 다양한 혜택 구조 반영
def parse_universal_content(soup_element):
    if not soup_element: return
    # 표 처리
    tables = soup_element.find_all('table')
    for t in tables:
        rows = t.find_all('tr')
        table_str_list = []
        for row in rows:
            cells = [ele.get_text(strip=True) for ele in row.find_all(['th', 'td'])]
            if cells: table_str_list.append(" | ".join(cells))
        if table_str_list:
            text_table = "\n\n[상세 표 데이터]\n\n" + "\n\n".join(table_str_list) + "\n\n"
            t.replace_with(text_table)
    # 리스트 처리
    dis = soup_element.find_all('div')
    for di in dis:
        dt = di.find("dt")
        dd = di.find("dd")
        if dt and dd:
            title = dt.get_text(strip=True)
            content = dd.get_text(separator="\n", strip=True)
            formatted_text = f"\n\n▶ [{title}]\n\n{content}\n\n"
            di.replace_with(formatted_text)

driver = create_driver()
all_data = []

for i in TARGET_RANGE:
    if i % 50 == 0:
        driver.quit()
        driver = create_driver()

    url = f"https://www.card-gorilla.com/card/detail/{i}"
    try:
        driver.get(url)
        # 카드 이름
        try:
            WebDriverWait(driver, 3).until(
                EC.presence_of_element_located((By.CSS_SELECTOR, "strong.card"))
            )
        except:
            print(f"[{i}] ✖ 페이지 없음 (Skip)")
            continue

        card_name = driver.find_element(By.CSS_SELECTOR, "strong.card").text.strip()
        # 동적 크롤링
        try:
            buttons = driver.find_elements(By.CSS_SELECTOR, "div.bnf_list dt, article.cmd_con.benefit dt, section.caution dt")
            for btn in buttons:
                try:
                    driver.execute_script("arguments[0].click();", btn)
                    time.sleep(0.1) # 최소한의 간격
                except: pass
            except: pass

            time.sleep(1.5) # 상세혜택 펼쳐보기 후 렌더링 대기
            soup = BeautifulSoup(driver.page_source, 'html.parser') # 데이터 추출

            # 혜택 영역
            benefit_area = soup.select_one("div.bnf_list, article.cmd_con.benefit")

            if benefit_area:
                parse_universal_content(benefit_area)
                raw_data = benefit_area.get_text(separator="\n", strip=True)
            else:
                raw_data = "해당 정보 없음"
```

상이한 웹사이트의 특성을 모두 반영하는 과정에서 불필요한 광고성 문구 등이 raw data 에 포함되게 되었으며 할인률, 적립 등에 따라 나누어지지 않은 상태였다. 이는 데이터 베이스에 즉시 적재하여 Query 할 수 없는 상태였기에, 고도화된 NLP 파이프라인 구축이 필수적이었다.

제 2 절 소상공인시장진흥공단 DB 활용

본 연구에서는 사용자 위치 기반 서비스 구현을 위해 소상공인시장진흥공단에서 제공하는 전국 상권정보 데이터를 활용하였다. 수집 범위는 전국 17 개 시·도 전체이며, 데이터는 지역별 csv 파일 형태로 구성되어 있다. 각 지역 파일은 동일한 스키마를 공유하고 있어, 이를 병합하면 약 280 만여 건의 전국 단위 통합 가맹점 데이터베이스가 된다.

본 데이터베이스는 원본 데이터의 방대한 속성 중, 카드 혜택 매칭과 지도 시각화에 필수적인 세 가지 주요 정보를 포함한다.

- i) 상점 식별 정보 : 카드 혜택 데이터의 브랜드 키워드와의 매칭에 필요하다. 프랜차이즈의 경우, '상호명'과 '지점명'을 결합하여 고유성을 확보한다.
- ii) 업종 분류 정보 : 특정 브랜드 대상이 아닌 포괄적 혜택 처리에 필수적이다. 특히 '상권업종소분류명'을 활용하여 매칭 정확도를 높인다.
- iii) 위치 정보 : 사용자의 모바일 기기 GPS 좌표와 상점 간의 거리 계산에 사용되어, 반경 내 접근 가능한 매장을 필터링하고 지도에 시각화 하는 데 사용된다.

제 3 장 분석 및 구조화 과정

제 1 절 LangChain 기반 구조화 파이프라인

비정형 텍스트를 데이터 베이스 Schema 에 적합한 JSON 형태로 변환하기 위해 LangChain 프레임워크에 Google 의 Gemini 2.5Flash 모델을 도입하였다. LangChain 은 LLM 을 활용한 애플리케이션 개발을 간소화하고, 복잡한 데이터 처리 워크플로우를 효율적으로 구성할 수 있게 해주는 도구이다. 본 연구에서는 단순 요약 시 발생할 수 있는 LLM Hallucination 이나 혜택 정보 누락을 방지하기 위해, 데이터 처리 과정을 3 단계로 세분화한 파이프라인을 설계하였다.

```
# 데이터 모델 Pydantic 구조
class CardBenefitDetail(BaseModel):
    merchant_name: str = Field(description="상호명 (예: 스타벅스, 이마트). 특정 상호가 없으면 업종명(예: 대중교통).")
    category: str = Field(description="대분류 (예: 푸드, 쇼핑, 교통, 주유, 통신, 공과금, 여행 등)")
    benefit_summary: str = Field(description="혜택 요약 (예: 10% 할인, 리터당 60원 적립)")
    conditions: str = Field(description="실적 조건. 없으면 '없음'")
    limit_info: str = Field(description="한도/횟수. 없으면 '제한없음'")
    is_location_based: bool = Field(description="지도에 찍을 수 있는 오프라인 장소인지 여부 (True/False)")

class CardBenefitList(BaseModel):
    benefits: List[CardBenefitDetail]
```

i) 노이즈 제거

입력된 raw data 에서 카드 혜택과 무관한 UI 요소, 광고 등을 제거하고, 카드 혜택 문맥만을 보존하도록 텍스트를 정제하였다.

```
# 노이즈 제거
cleaner_template = """
당신은 데이터 전처리 전문가입니다.
아래 [Raw Text]는 웹사이트에서 크롤링한 데이터로 광고와 메뉴 등 노이즈가 섞여 있습니다.

[할 일]
1. **{card_name}** 카드와 관련된 **혜택 정보, 실적 조건, 유의사항**만 남기세요.
2. 광고, 내비게이션 메뉴, 다른 카드 추천 등은 삭제하세요.
3. 내용을 요약하지 말고, **원문의 문장을 그대로 보존**하여 정리하세요.

[Raw Text]:
{raw_text}

[Cleaned Text]:
"""
```

ii) 정보 추출

정제된 텍스트를 분석하여 데이터베이스 필드에 매핑 될 핵심 Entity 를 추출하였다.

㉠ Merchant_name: 혜택이 적용되는 가맹점명 혹은 브랜드명

㉡ Benefit summary: 혜택의 구체적인 내용

㉢ Conditions: 혜택 적용을 위한 실적 등의 조건

㉣ Limit_info: 혜택 적용 한도 및 횟수 제한

```
# 혜택 분석
analyzer_template = """
당신은 혜택 분석가입니다.
[Cleaned Text]를 읽고, 어떤 혜택들이 있는지 목록으로 정리하세요.

[할 일]
1. 텍스트에 숨겨진 모든 혜택을 하나씩 분리하여 나열하세요.
2. 각 혜택의 '**대상 가맹점(Merchant)**'과 '**혜택 내용(Summary)**', '**조건(Condition)**'을 명확히 파악하세요.
3. 아직 JSON으로 만들지 말고, **사람이 읽기 좋은 텍스트 리스트** 형태로 출력하세요.

[Cleaned Text]:
{cleaned_text}

[Benefit List]:
"""
```

iii) 가맹점의 온/오프라인 여부 확인 및 구조화

추출된 혜택이 지도상의 오프라인 상가와 연동 가능한지를 논리적으로 판단하여 is_location_based 칼럼을 생성하고, 최종 결과를 Pydantic 라이브러리를 통해 JSON 형태로 구조화하였다.

```

# 지도 판단 및 구조화
mapper_template = """
당신은 GIS(지리정보시스템) 및 데이터 구조화 전문가입니다.
앞서 분석된 [Benefit List]를 보고 최종 데이터를 생성하세요.

[핵심 판단 로직: is_location_based]
각 혜택의 가맹점이 "*"사용자가 지도 앱(네이버 지도 등)에서 검색해서 찾아갈 수 있는 물리적 장소인가?"**를 판단하세요.

* **True (지도 O):** 식당, 카페, 편의점, 마트, 주유소, 병원, 약국, 영화관, 놀이공원, 공항 라운지, 백화점, 아울렛
* **False (지도 X):** - 온라인 쇼핑 (쿠팡, G마켓, 배달의민족 결제 등)
- 정기결제/구독 (넷플릭스, 유튜브, 멜론, 아파트 관리비, 도시가스, 통신요금)
- 간편결제 (네이버페이, 카카오페이 자체 혜택)
- 해외 결제 (해외 직구 포함)

[Benefit List]:
{analyzed_text}

[출력 형식]:
{format_instructions}
"""

```

LangChain 을 통해 LLM 을 통과한 structured data 의 신뢰성을 검증하기 위해, 크롤링 원본 raw data 와 수치적 비교 검증을 하였다. 이는 raw data 와 structured data 의 숫자 집합과 텍스트의 길이를 수치적으로 비교함으로써 LLM Hallucination 이나 혜택 정보 누락을 평가하고자 한 것이다.

```

for card_id, benefits in grouped_db.items():
    raw_text = str(raw_map.get(card_id, ""))

    # 숫자 일치도 검사
    # : raw data에 있는 숫자들이 structured data에도 있는지 확인
    raw_numbers = set(re.findall(r'\d+(?:,\d+)?', raw_text))
    extracted_text_concat = " ".join(benefits)
    db_numbers = set(re.findall(r'\d+(?:,\d+)?', extracted_text_concat))
    # : raw data에 없는데 structured data에 등장한 숫자가 있는지 확인
    hallucination_nums = db_numbers - raw_numbers

    # 키워드 생존율
    # : raw data의 핵심 키워드가 structured data에 얼마나 포함되었는지 확인
    raw_key_count = raw_text.count('%') + raw_text.count('원')
    db_key_count = extracted_text_concat.count('%') + extracted_text_concat.count('원')

    recovery_rate = (db_key_count / raw_key_count * 100) if raw_key_count > 0 else 0

    # 길이 비율
    # raw data 길이 대비 structured data 텍스트 양이 너무 적지 않은지 확인
    len_ratio = len(extracted_text_concat) / len(raw_text) if len(raw_text) > 0 else 0

```

그 결과, 최초의 단일 프롬프트에 비해 3 단계 프롬프트에서 무결성 의심 카드가 42 개 감소하였고, 의심되는 카드 중 단 1 개만이 혜택 누락 문제가 있었음이 확인되었다. 카드 혜택 DB 에서 이를 수정함으로써 카드혜택 데이터셋의 무결성을 확보하였다.

제 2 절 LangChain 기반 매칭 칼럼 생성

구축된 카드 혜택 DB를 실제 위치 기반 서비스에 적용하기 위해, 소상공인진흥공단의 전국 280만여 건의 상가 데이터와 카드 혜택 정보를 연결하는 매칭 알고리즘을 개발하였다. 초기에는 NER 모델을 고려하였으나, 상호명의 다양성과 모호성을 반영하는 데 한계가 있어 LangChain을 활용하여 키워드 추출과 규칙 기반 교차 검증을 결합한 매칭 알고리즘을 설계하게 되었다.

```
# Pydantic 출력 표시 설정
class BrandCategoryKey(BaseModel):
    brand_key: str = Field(description="분석된 브랜드 표준명 (또는 특수 코드 '전세'). 없으면 빈 문자열.")
    category_key: str = Field(description="분류된 카테고리 코드 (19개 중 필수 선택).")

# 최종 시스템 프롬프트 (Final Ver.)
SYSTEM_INSTRUCTION = """
당신은 상점 데이터를 분석하여 신용카드 혜택 매칭 시스템에 필요한 핵심 인덱스의 "brand_key"와 "category_key"를 추출하는 데이터 분석 전문가입니다.

당신의 역할은 상가 데이터를 분석하여 브랜드 단위의 명명 매칭과 업종 단위의 분류 매칭이 모두 가능하도록 정확한 JSON 데이터를 생성하는 것입니다. 아래의 규칙을 엄격히 준수하십시오.

### 1. 출력 형식 (JSON Only)
반드시 올바른 마크다운 태그 없이 아래의 JSON 포맷으로만 출력하십시오.
{
  "brand_key": "분석된 브랜드 표준명 (또는 특수 코드)",
  "category_key": "분류된 카테고리 코드 (필수 선택)"
}

### 2. brand_key 생성 규칙 (정원 매칭용)
카드 혜택 데이터에 브랜드명이 명시된 경우(1순위 매칭)를 위해 사용합니다.

- **대상:** 전국적으로 사용되는 프랜차이즈, 대형 유통 채널, 플랫폼 기반인 경우에만 입력합니다. (개인 상점, 동네 일반 병원/약국, 일반 음식점 등은 공백 "" 처리)
- **지명명 제거:** 상호명에서 지명, 지역, 옵션 정보를 연백이 제거됩니다. (예: "스타벅스 강남점" -> "스타벅스", "이마트24 용인점" -> "이마트24")
- **표준화:** 공식 브랜드명으로 통일합니다. (예: "지이스25" -> "GS25", "파리바게트" -> "파리바게트", "농협하나로마트" -> "하나로마트")
- **[특수 규칙] 전세 가맹점 처리:** 입력된 상호명이 특정 브랜드가 아니라 '전세' 또는 '가맹점', '국내 가맹점', '외국 가맹점', '전 업종' 등 범용 개념을 의미할 경우, brand_key에 반드시 '전세'라고 입력하십시오.
```

```

### 3. category_key 분류 규칙 (보통 매장용)
카드 혜택이 적용 브랜드가 아닌 업종 전체(예: "모든 편의점 판매")에 적용될 때(2순위 매칭)를 위해 사용됩니다.
반드시 아래 19가지 코드 중 하나를 선택해야 합니다.

**[하위권 카테고리 코드 목록]**
[카페/음료, 베이커리/디저트, 푸드/외식, 패스트푸드/배달음,
편의점, 대형마트/슈퍼, 온라인쇼핑, 패션/잡화, H&M/뉴타,
주유/충전, 교통/모빌리티, 여행/숙박, 문화/여가,
방문/악곡, 교육/육아, 통신/엔터테인먼트, 공공금/생물,
금융/보험, 기타/서비스]

**[보류 우선순위 및 가이드]**
상권인종대분류(예: 음식, 소매)보다 **실제 서비스의 성격**을 좌우선으로 반영하여 세분화합니다.

1. **식음료(F&B) 세분화 원칙 (매우 중요)**
- **카페/음료:** 커피전문점, 주스, 과일 등 유자 음료 매장 (일반 음식점 제외).
- **베이커리/디저트:** 빵집(파리바게트, 푸레쥬르), 도넛(단란), 아이스크림(베스킨라빈스), 떡집.
- **패스트푸드/배달음:** 피자(맥도날드), **치킨(교촌, BBQ 등), 피자(도미노 등) 프랜차이즈 및 **배달앱(배달의민족, 요기요)**
- **푸드/외식:** 위 3가지에 해당하지 않는 모든 일반 식당, 주점, 패밀리레스토랑(아름뽕, 뽕스).

2. **소매/유통 세분화 원칙**
- **편의점:** GS25, CU, 세븐일레븐, 이마트24, 미니ストップ 등.
- **대형마트/슈퍼:** 이마트, 홈플러스, 롯데마트, **농협하나로마트, 트레이디스, 롯데슈퍼, GS The Fresh.
- **온라인쇼핑:** 쿠팡, 11번가, G마켓, **중소형(CJ오쇼핑 등)** 온라인 서점.
- **H&M/뉴타:** 올리브영, 굿즈, 알라블라, 미용실, 화장품 매장.
- **패션/잡화:** 백화점, 아울렛, 면세점, 다이소, 의류/신발 매장.

3. **생활/이동/육아 업종 분류**
- **주유/충전:** 주유소(SK, GS, S-OIL 등), LPG충전소, 전기차충전소.
- **교통/모빌리티:** 버스/지하철/역시/KT, **렌터카, 포기, 주차장**, 하이패스.
- **여행/숙박:** 여행, 모텔, 연립형, 묵어, 산후조리원, 요양병원.
- **교육/육아:** 학원, 학습지, **유치원/어린이집**, **장부지원 배우자 지정기관**.
- **통신/엔터테인먼트:** 통신사요금(SKT/KT/LGU), 청수기/가전 렌탈.

4. **기타 예외 처리**
- **차량 정비:** 스피드메이트, 에너카렌드 등은 "기타/서비스" (단, 주유소 내 정비소와 연결치 않으면 주유/충전 불가).
- **포인트/멤버십:** L.POINT, OK Cashbag 등은 "기타/서비스".
- **보류 불가:** 위 카테고리에 명확히 속하지 않는 경우 "기타/서비스".

4. **기타 예외 처리**
- **차량 정비:** 스피드메이트, 에너카렌드 등은 "기타/서비스" (단, 주유소 내 정비소와 연결치 않으면 주유/충전 불가).
- **포인트/멤버십:** L.POINT, OK Cashbag 등은 "기타/서비스".
- **보류 불가:** 위 카테고리에 명확히 속하지 않는 경우 "기타/서비스".

### 4. 분석 예시 (Fee-Shot)
- "GS25 편의점내점" -> {"brand_key": "GS25", "category_key": "편의점"}
- "스타벅스 강남점" -> {"brand_key": "스타벅스", "category_key": "카페/음료"}
- "교촌치킨 용매점" -> {"brand_key": "교촌치킨", "category_key": "패스트푸드/배달음"}
- "김밥천국 본점" -> {"brand_key": "", "category_key": "푸드/외식"}
- "국내 전 여행점" -> {"brand_key": "전체", "category_key": "기타/서비스"}
- "농림어업관련" -> {"brand_key": "한미식품", "category_key": "대형마트/슈퍼"}
- "구멍" -> {"brand_key": "구멍", "category_key": "온라인쇼핑"}
- "SK에너지" -> {"brand_key": "SK에너지", "category_key": "주유/충전"}
- "시원디카" -> {"brand_key": "시원디카", "category_key": "교통/모빌리티"}
- "아이링크 배우자" -> {"brand_key": "", "category_key": "교육/육아"}
...

PROMPT_TEMPLATE = """
[입력 상세 데이터]
{input_data}

{format_instructions}
"""

```

(프롬프트)

카드 혜택의 적용 범위는 특정 업종에 대해서도 일부 브랜드에 한정되거나, 포괄적인 업종에 적용되는 두 가지 유형으로 나뉜다. 이를 상점 데이터에 정확히 매핑하기 위해 Brand_Key와 Category_Key로 이원화된 매칭 전략을 수립하였다.

- i) Brand_Key: 카드 혜택 DB에서 '스타벅스', 'GS25', '이마트'와 같은 고유 브랜드명을 LangChain을 통해 추출하여 리스트업하였다. 지역별 상가 데이터셋에서는 상호명을 분석하여 해당 리스트에 포함될 경우 Brand_Key 칼럼에 표준화된 브랜드명을 태깅하도록 하였다.

- ii) Category_Key: '전국 편의점 10% 할인'과 같은 범용 혜택의 처리를 위해, 상가 데이터셋의 상권업종대분류명 및 소분류명을 분석하여 카드 혜택 DB 와 매칭되도록 한 업종 키워드를 Category_Key 칼럼에 부여하였다.

단순 텍스트 매칭 시 발생할 수 있는 오류 해결을 위해 상호명과 업종 분류 코드를 교차 검증하는 로직을 구현하였다.

상호명 matched_brand		상호명 matched_brand	
고대어학원	어학원	닥터.지동물병원	동물병원
크리스마스호텔	호텔	경인가구마트	마트
테탈스파치과의원	스파	미니스톱영중운서점	서점
커피전도사의집	커피	세븐일레븐검암풍림점	세븐일레븐
하얀나무미술학원	학원	웨라pc방	pc방

(단순 텍스트 매칭 시 발생 가능한 오류)

제 3 절 SQLite 및 JS 기반 매장 매칭 및 지도 데이터 연계

본 시스템의 백엔드 아키텍처는 경량화된 로컬 환경에서 대용량 데이터를 효율적으로 처리하고 신속한 응답 속도를 보장하기 위해 Node.js 런타임과 SQLite 데이터베이스를 기반으로 구축하였다. 데이터베이스의 접근 계층에는 better-sqlite3를 채택하여 Node.js 환경에서의 데이터 처리 효율을 높였으며, 빈번한 읽기 및 쓰기작업이 발생하는 카드 추천 기능의 특성을 고려하여, DB의 저널링 모드를 WAL 방식으로 설정함으로써 동시성 제어성능을 강화하고 쓰기작업 시의 블로킹 현상을 최소화하였다.

앱의 핵심기능인 위치기반 매장 매칭과 지도 데이터의 연계를 구현하기 위해서는, 수만 건 이상의 매장 데이터 내에서 사용자 위치 주변에 존재하는 매장만을 선별하는 과정을 거쳐야만 한다. 이를 위하여, 본 연구에서는 SQL 쿼리 레벨의 1차

공간 필터링과 JS 엔진 단계의 2차 정밀 거리 계산을 결합한 하이브리드 매칭 알고리즘을 설계하였다.

1단계에서는 사용자의 현재 위치, 또는 사용자가 지정한 위치를 위도와 경도의 형태로 받아, 그것을 중심으로 조회반경에 해당하는 사각 영역의 바운딩 박스를 계산한다. 이때, 위도 1도당 거리를 약 111.32km로 고정하고, 경도 1도당 거리는 위도에 따라 변화하는 기하학적 특성을 반영하여 코사인 함수로 보정하여, 검색 영역의 정확도를 확보하였다. 해당 과정을 통과한 데이터에 대해서는 2단계 정밀 검증을 수행한다. 이 과정에서는 JS의 Haversine 공식을 적용하여 사용자와 각 매장 간의 실제 거리를 미터 단위로 정밀하게 계산한다. 단순히 직선거리로 계산하는 것이 아닌 지구의 곡률을 반영한 Haversine 계산을 통해 500m 반경 이내에 실제로 존재하는 매장만을 최종적으로 필터링하며, 결과 데이터는 거리순으로 정렬되어 클라이언트에 전달된다.

```
Search Here triggered lat=37.263185188364936 lon=127.0331834256649 categories=
Fetched 2534 stores for lat=37.263185188364936 lon=127.0331834256649 radius=500 categories=all
Store[0]=107007053:더케이웨딩컨벤션부페@37.2632862000974,127.033065089924 category=OTHER
Store[1]=107173712:화이트바른이치과교정과치과의원@37.2632862000974,127.033065089924 category=HEALTHCARE
Store[2]=107247807:에스안과의원@37.2632862000974,127.033065089924 category=HEALTHCARE
Store[3]=107564606:아비쥬의원수원@37.2632862000974,127.033065089924 category=HEALTHCARE
Store[4]=106991125:샐러디@37.2628627385726,127.032963444131 category=DINING
```

(사용자가 지정한 위치를 기준, 주변 매장의 데이터를 클라이언트가 받은 모습)

제 4 장 시스템 구현 결과

제 1 절 앱 개발(프론트엔드·백엔드)

본 연구의 시스템은 사용자 접점인 안드로이드 모바일 앱과 데이터 처리를 담당하는 로컬 백엔드 서버로 구성된 클라이언트-서버 아키텍처로 구현되었다. 프론트엔드인 안드로이드 애플리케이션은 최신 안드로이드 개발의 표준인 Jetpack Compose UI 킷과 Kotlin 을 사용하여 100% 선언형 UI 로 개발되었다. 또한, Kotlin Coroutines 와 StateFlow 를 도입하여, 데이터 처리 중에도 UI 가 멈추지 않고 반응적으로 동작하도록 구현하였다. 네트워크 통신 계층은 Retrofit 과 OkHttp 라이브러리를 기반으로 구축되었으며, 일반적인 REST API 호출뿐만 아니라 실시간 추천 결과 수신을 위한 SSE 스트리밍 클라이언트를 포함하여 서버의 응답을 지연 없이 사용자에게 전달할 수 있는 체계를 갖추었다.

백엔드 시스템은 Node.js와 Express 프레임워크를 기반으로 설계되었으며, 로컬 환경에서의 설치 및 실행의 편의성을 고려하여 경량화된 구조를 채택하였다. 이러한 백엔드의 모듈에는 카드 추천 시스템과 상술한 위치 계산 및 매장 매칭 모듈이 포함된다.

제 2절 UI/UX 및 위치 기반 추천 기능 구현

UI/UX의 경우, 지도 중심의 탐색과 신속한 결과 확인에 중점을 두고 구현하였다. 위치 수집을 위해 Google Play Services의 FusedLocationProviderClient를 사용하여 현재 위치 데이터를 확보하고, 이 데이터는 주변 매장을 매칭 모듈에 전달된다. 지도 시각화는 Google Maps Compose 라이브러리를 활용하였으며, 좁은 지역에 밀집한 매장 정보를 효과적으로 표현하기 위해 클러스터링 알고리즘을 적용

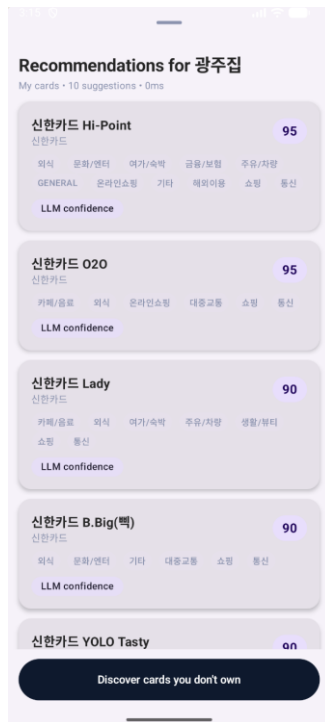


하여 지도의 가독성을 높였다. 또한 매장 카테고리에 따라 차별화된 아이콘을 렌더링하여 시각적 인지도를 개선하였다.



(사용자 주변의 위치에 대한 군집화된 매장 마커)

상술한 카드 추천 시스템의 작동은, 우선 사용자가 보유한 카드 중에서 선택된 매장 데이터의 brand_key와 혜택의 내용을 비교하는 과정을 통해 현재 위치한 매장에서 명시적 혜택을 제공하는 카드를 우선적으로 매칭하는 과정이 수행된다. 이후 Gemini API와 연동하여 선택된 매장의 업종 분류와 각 카드의 혜택을 분석하고 점수를 산출하는 과정이 이어지며, 네트워크 오류나 응답 지연에 대비한 휴리스틱 알고리즘이 최종적으로 적용되어 시스템의 안정성을 보장한다.



(임의의 선택된 매장에 대한 카드 추천 시스템의 결과)

또한, 사용자가 보유하지 않은 카드 중에서 매장 데이터와의 매칭 결과를 볼 수 있는 기능을 구현하여, 사용자가 보유한 카드 중 선택된 매장에서의 사용을 추천할 만한 카드가 없거나 사용자가 해당 매장에서의 사용에 최적화된 다른 카드를 탐색하기 원하는 경우를 감안하였다.



(사용자가 보유하지 않은 카드에 대하여 추천 시스템을 적용한 결과)

제 5 장 결론

제 1 절 연구 결과 종합 및 시사점

본 연구에서는 비정형 텍스트로 웹 상에 존재하는 신용카드 혜택 정보를 수집하여 LLM(Large Language Model)기반의 LangChain 프레임워크를 통해 정형 데이터로 구조화하고, 이를 전국단위의 소상공인 상권 정보와 결합하여 사용자 위치 기반의 맞춤형 혜택 추천 시스템을 구현하였다. 주요 시사점은 다음과 같다.

첫째, 온라인 상의 텍스트 데이터인 '카드 혜택 정보'와 '오프라인 물리적 공간 데이터인 '상권 정보'를 매칭 키 알고리즘으로 결합하는 데 있어, 고비용의 GIS인 프라 대신 SQLite 기반의 경량화된 아키텍처와 하이브리드 알고리즘을 적용하였다. 이는 모바일 환경에서의 비용 효율적인 핀테크 모델을 제시하였다는 점에서 의의가 있다.

둘째, 본 시스템은 LLM에만 의존하지 않고, 규칙기반 매칭 - 생성형 AI 분석 - 휴리스틱 보정 으로 이어지는 추천 로직을 구현하여, 금융서비스의 핵심 가치인 신뢰성과 가용성을 담보한다.

셋째, 본 시스템은 복잡한 혜택 조건으로 인해 소비자가 감수해야 했던 정보 탐색 비용을 획기적으로 낮춘다. 사용자가 현재 위치 근방의 상점에서 즉시 제공 받을 수 있는 혜택을 시각적으로 제공함으로써, 소비자의 실질적인 가처분 소득 증대 효과를 유도하고 합리적인 금융 생활을 지원하는 사회적 효용을 갖는다.

제 2 절 한계점 및 향후 연구 방향

본 연구는 위치 기반 신용카드 추천 시스템의 프로토타입을 성공적으로 구현하였으나, 이를 개선하기 위한 후속 연구가 요구된다.

첫째, 데이터의 실시간 동기화 문제 : 카드사의 혜택 변경 등 데이터의 실시간 변동 사항을 즉각 반영하는 데 한계가 존재한다. 향후 연구에서는 앱 실행 시 변경된 데이터만 선별하여 다운로드하는 프로토콜을 구현하거나, 서버와의 하이브리드 통신 모델을 도입하는 등의 방식 적용이 필요할 것으로 예상된다.

둘째, 복잡한 실적 조건 계산의 한계 : 현재 시스템은 개인화된 조건 충족 여부를 실시간으로 판단하기에는 사용자 금융 데이터 접근 권한의 제약이 따른다. 이를 해결하기 위해 사용자가 직접 실적을 입력하거나, 개인의 카드 사용 내역을 안전하게 연동 가능하게 하는 정밀한 연구가 필요할 것이다.



셋째, 본 연구에서 개인화 추천 알고리즘과 조건 충족 여부 파악을 위한 영수증 OCR 기술 도입의 논의도 있었으나, 제한된 연구 기간 내에 위치 기반 매칭 시스템의 안정성을 확보하는 것을 최우선 과제로 삼으며 실제 구현 단계에서는 제외되었다. 향후 연구에서는 이러한 기술을 알고리즘 고도화 과정에 반영한 사용자 편의성 개선을 제언한다.

결과적으로 본 시스템이 고도화되어 사용된다면, 소비자는 더 이상 복잡한 혜택을 기억할 필요 없이 결제 순간에 최적의 의사결정을 내릴 수 있게 될 것이며, 이는 건전한 신용카드 소비 문화를 정착시키는 데 기여할 것으로 기대된다.



참고문헌

- 금융위원회. (2.25). 여전업권 CEO 간담회 모두 말씀 [보도 자료].
- 여은정&전병준. (2016). 국내 신용카드사 제공 서비스 분석과 시사점: 할인 및 적립 혜택을 중심으로. 금융안정연구, 17(2), 51-72.
- 최성철. (2024). 랭체인(LangChain)이란 무엇인가? 삼성SDS.
<https://www.samsungsds.com/kr/insights/what-is-langchain.html>

